Téma 11: Sdílené proměnné, komunikace a profiling

Teoretický základ

Tento blok se věnuje čistotě a rychlosti kódu – důležitý krok od začátečníka k pokročilejšímu programátorovi.

- **Sdílené (globální) proměnné:** Komunikovat s funkcemi prostřednictvím globálních proměnných, které se mění odněkud (tzv. "vedlejší efekty" neboli side-effects), je velmi nebezpečná praxe (tzv. anti-pattern). Těžko se ladí. Funkce by měly přijímat hodnoty vstupem (parametry) a posílat výstup (return).
- **Profiling a optimalizace:** Než začneme kód urychlovat, musíme ho změřit. Modul `timeit` měří dobu běhu krátkého bloku. Pro velké skripty měříme funkce modulem `cProfile`.
- **Pokročilé typy (collections):** Třída `Counter` z modulu `collections` dokáže efektivně na jeden řádek sečíst frekvenci všech prvků v seznamu.

Základní koncept k zamyšlení: Časová složitost (Big O Notation)

Pro pochopení výkonu programu je nutné znát pojem "Time Complexity". Příkladem může být markantní rozdíl ve vyhledávání prvku v seznamu (List) a hledáním přes klíč ve slovníku (Dictionary). Zatímco u seznamu musíme projít prvek po prvku (lineární složitost O(n)), slovník umožňuje díky hashování klíčů okamžitý přístup k hodnotě (konstantní složitost O(1)). Slovník je tedy pro časté vyhledávání nepoměrně rychlejší.

Úkol 1 (Lehká obtížnost)

Zadání úkolu: Dostali jste seznam: `["jablko", "hruška", "jablko", "banán", "jablko"]`. V rámci efektivní algoritmizace nepište pomalý for cyklus. Využijte vysoce optimalizovanou a zrychlenou třídu `Counter` z knihovny `collections`, pomocí které jedním příkazem spočítáte, kolikrát které slovo v seznamu je.

Řešení úkolu 1

```
from collections import Counter

kosik_ovoce = ["jablko", "hruška", "jablko", "banán", "jablko"]

# Counter okamžitě a optimalizovaně analyzuje frekvenci prvků
# a vrátí speciální slovník.
pocitadlo = Counter(kosik_ovoce)

print(pocitadlo)
# Lze přímo dotazovat i konkrétní prvek jako u klasického slovníku
print(f"Samotných jablek je přesně: {pocitadlo['jablko']}")
```

Úkol 2 (Střední obtížnost)

Zadání úkolu: Udělejte jednoduché rychlostní měření (profiling) vašeho softwaru. Z knihovny `time` naimportujte vysoce přesnou funkci `perf_counter()`, odečtěte hrubý čas před započetím milionového prázdného for cyklu a porovnejte ho s časem po skončení oběhů. Vypište zjištěnou dobu.

Řešení úkolu 2

```
import time

# Uložíme si přesnou referenční hodnotu času "T0"
start_cas = time.perf_counter()

# Na chvíli zatížíme procesor (milion průchodů bez jakékoliv práce)
for   in range(1_000_000):
    pass

# Uložíme čas po doběhnutí "T1"
konec_cas = time.perf_counter()

doba zpracovani = konec cas - start cas
print(f"Výpočet trval celkem: {doba_zpracovani:.5f} sekund.")
```

Úkol 3 (Těžká obtížnost)

Zadání úkolu: Jste unaveni psaním sčítací funkce, která přijímá pokaždé jen dva vstupní parametry (a, b). Pokročilé Python funkce umožňují zpracovat flexibilní počet vstupů. Definujte funkci využívající zápis `*args`, aby v parametrech přijala naprosto libovolný počet argumentů zaráz, vnitřně je sečetla a výsledek vrátila.

Řešení úkolu 3

```
# Hvězdička v parametrech zabalí všechny dodané hodnoty do jedné n-tice
(Tuple)
def secti_vsechno(*args):

    # Nyní v args máme např. Tuple (5, 10, 15, 20)
    # Na ten aplikujeme zrychlenou vestavěnou C funkci sum()
    return sum(args)

# Lze zavolat naprosto s jakýmkoliv počtem čísel!
vysledek dva = secti vsechno(5, 10)
vysledek_sest = secti_vsechno(1, 2, 3, 4, 5, 6)

print(f"Součet dvou: {vysledek_dva}, Součet šesti: {vysledek_sest}")
```

Úkol 4 (Lehká obtížnost - dodatek)

Zadání úkolu: (List Comprehension). Chcete získat pole druhých mocnin čísel od 1 do 10. Místo klasického for cyklu, definice prázdného pole a volání zdlouhavého `append()` použijte optimalizovaný zápis tzv. generování seznamu (List comprehension), u kterého vznikne výsledné pole okamžitě zevnitř na jednom řádku.

Řešení úkolu 4

```
# Klasicky bychom psali 3 řádky. List comprehension to zvládne vnitřně a velmi rychle:
# Zápis: [ VÝSLEDEK_VZOREC cyklus PODMÍNKA ]

mocniny = [x**2 for x in range(1, 11)]

print(f"Získané dynamické pole mocnin: {mocniny}")
```

Úkol 5 (Střední obtížnost - dodatek)

Zadání úkolu: K algoritmickému urychlení se zřídkakdy definují jednorázové bezejmenné funkce (`lambda`). Společně s vestavěným nástrojem `filter()` si připravte pole `[1, 5, 8, 12, 15]` a nechte do nového pole odfiltrovat výhradně ta čísla, která jsou sudá.

Řešení úkolu 5

```
cisla = [1, 5, 8, 12, 15]

# Funkce filter přijme jako první parametr "vyhodnocovací stroj"
# (v našem případě jednorázovou Lambda funkci, ptající se na x % 2 == 0),
# a jako druhý parametr balík dat (cisla).
# Výsledek musíme přetypovat na živý list().
suda_cisla = list(filter(lambda x: x % 2 == 0, cisla))

print(f"Profiltrováno! Sudé hodnoty jsou: {suda_cisla}")
```

Úkol 6 (Těžká obtížnost - dodatek)

Zadání úkolu: (Vlastní generátor - the `yield` keyword). Napište funkci, která je nekonečná, ale přesto nezboří a nezahluší paměť RAM. Použijte klíčové slovo `yield` k postupnému navracení přirozených čísel. Funkce se při vyvolání yielda zastaví, stav zamrazí a pošle výsledek ven. Při dalším průchodu navazuje přesně tam, kde přestala.

Řešení úkolu 6

```
# Generátor nevrací data přesně přes return, protože return zahodí stav a
skončí.
def nekonecna_cisla():
    n = 1
    while True:
        # Yield vydá hodnotu, ale vnitřní while běží dál
        yield n
        n += 1

generovane = nekonecna_cisla()

# Zavoláme speciální funkci next(), ta požádá generátor o probuzení na
další krok
print(next(generovane))  # Probere se, dá 1, usne
print(next(generovane))  # Probere se, zvětší paměť, dá 2, usne
print(next(generovane))  # 3
```